JSPM'S

RAJARSHI SHAHU COLLEGE OF ENGINEERING

TATHAWADE, PUNE-411033

DEPARTMENT OF COMPUTER ENGINEERING

Mini Project

on

"California House Price Prediction & Real Estate Valuation System"

-Submitted by-

Harshdeep Singh

Roll No: CS3149

Class and Division: TY Div A

Course: Machine Learning (Course Code: 310246)

Academic Year: 2026-27

(Semester-I)

REPORT CONTENTS

In accordance with SPPU Computer Engineering Mini Project Guidelines

TABLE OF CONTENTS

1. Title & Details ..... **Page 3**

2. Abstract ..... **Page 3**

3. Introduction ..... **Page 4**

3.1 Background & Motivation ..... **Page 4**

3.2 The Need for Automated Valuation Models (AVMs) ..... **Page 4**

3.3 Curriculum Alignment with SPPU Machine Learning Lab ..... **Page 4**

4. Problem Statement ..... **Page 5**

5. Objectives ..... **Page 5**

6. Data set and Libraries required ..... **Page 6**

6.1 California Housing Dataset ..... **Page 6**

6.2 Engineered Features ..... **Page 6**

6.3 Libraries & Hardware Stack ..... **Page 6**

7. Approach / Methodology ..... **Page 7**

7.1 End-to-End Pipeline (D.A.T.A. Protocol) ..... **Page 7**

7.2 Mathematical Foundations ..... **Page 7**

8. Implementation ..... **Page 8**

9. Results & Experimental Analysis ..... **Page 9**

9.1 Regression Benchmark Results ..... **Page 9**

9.2 Feature Importance Analysis ..... **Page 9**

9.3 Interactive Web Application Deployment ..... **Page 10**

10. Conclusion & References ..... **Page 10**

1. Title

Project Title: California House Price Prediction & Real Estate Valuation System

Academic Department: Department of Computer Engineering, JSPM's Rajarshi Shahu College of Engineering, Tathawade, Pune-411033

Affiliated University: Savitribai Phule Pune University (SPPU)

Class & Division: Third Year Computer Engineering (TY Div A), Academic Year 2026-27 (Semester-I)

Course & Lab: Machine Learning (Course Code: 310246)

Candidate: Harshdeep Singh (Roll No: CS3149)

2. Abstract

In modern macroeconomics and municipal finance, real estate appraisal is vital for mortgage underwriting, property taxation, and investment portfolios. Traditional manual appraisal methodologies are prone to subjective human bias, localized transaction infrequency, and poor scalability. Furthermore, standard linear econometric techniques struggle to model complex spatial autocorrelation and non-linear interactions between demographic indicators and physical housing characteristics.

This mini project presents an end-to-end, mathematically grounded, reproducible Machine Learning pipeline: the **California House Price Prediction & Real Estate Valuation System**. Built upon the benchmark California Housing dataset (20,640 census block groups derived from the 1990 U.S. Census), the project implements the structured **D.A.T.A. Protocol (Discover → Analyze → Train → Assess)** to enforce pipeline isolation, prevent data leakage, and ensure model robustness.

The project benchmarks six distinct machine learning algorithms across standardized 5-fold cross-validation and a 20% holdout test split:

1. **Median Baseline Regressor:** Empirical lower bound (RMSE = 0.3558, R^2 = -0.0042).
2. **Multiple Linear Regression:** Parametric OLS baseline (RMSE = 0.2142, R^2 = 0.6361).
3. **Ridge Regression:** L2-penalized regularizer mitigating multicollinearity (RMSE = 0.2142, R^2 = 0.6361).
4. **Lasso Regression:** L1-penalized sparse regularizer (RMSE = 0.2816, R^2 = 0.3709).
5. **Random Forest Regressor:** Bagging ensemble of 100 deep decision trees (RMSE = 0.1611, R^2 = 0.7941).
6. **XGBoost Regressor:** Second-order gradient boosted decision trees (RMSE = 0.1424, R^2 = 0.8391).

> **LEAK-FREE D.A.T.A. PROTOCOL PRINCIPLE**

> By enforcing a strict boundary between feature extraction, standard scaling, and model fitting, the pipeline guarantees that test set evaluations remain strictly out-of-sample. Target skewness is addressed using natural logarithmic transformation ($\ln(1 + y)$), ensuring stable optimization and unbiased error distributions.

3. Introduction

3.1 Background & Motivation

Housing valuation represents one of the largest capital asset segments in contemporary society. In real estate markets, property prices are dictated by a complex combination of micro-demographics (e.g., median household income, block occupancy), physical infrastructure (e.g., average room count, building age), and macroeconomic trends. Accurate property price estimation protects financial institutions from over-leveraging and empowers home buyers with data-driven transparency.

3.2 The Need for Automated Valuation Models (AVMs)

Automated Valuation Models (AVMs) leverage statistical learning to process continuous real estate indicators and output instant, reliable home appraisals. Traditional appraisal reports rely on manual comparables ("comps"), which are slow and subjective. Machine learning enables real-time valuation across thousands of properties simultaneously, incorporating multi-dimensional spatial dynamics that linear equations fail to capture.

3.3 Curriculum Alignment with SPPU Machine Learning Lab

This mini project holistically integrates core algorithmic paradigms prescribed in the Savitribai Phule Pune University (SPPU) Third Year Computer Engineering Machine Learning syllabus:

Curricular Topic	Implemented Technique	Curricular Focus & Scope
Linear Models & OLS	Multiple Linear Regression	Parametric estimation, closed-form normal equations, and coefficient significance.
Regularization	Ridge (L2) & Lasso (L1)	Penalized cost optimization; combating collinearity and coefficient shrinkage.
Ensemble Learning (Bagging)	Random Forest Regressor	Bootstrap aggregation of 100 decorrelated trees; non-linear boundary estimation.
Ensemble Learning (Boosting)	XGBoost Regressor	2nd-order Taylor expansion loss minimization, shrinkage rate, and tree depth tuning.
Data Preprocessing	StandardScaler, Log-Transforms	Normalizing heteroscedastic feature scales and inverting log-target distributions.
Model Evaluation	5-Fold Cross-Validation, RMSE, MAE, R^2	Out-of-sample generalization metrics, residual normality checks, and feature ranking.

Curricular Topic	Implemented Technique	Curricular Focus & Scope
Full-Stack Deployment	Interactive Streamlit GUI	Real-time user inference, PyDeck GIS mapping, and batch CSV valuation engine.

4. Problem Statement

Predicting real estate prices from multi-source census and geographic data presents acute computational and statistical challenges:

- **Non-Linear Spatial Autocorrelation:** Latitude and longitude coordinates exhibit severe non-linear pricing effects (e.g., steep price premiums along coastal California pockets compared to the Central Valley). Simple linear models cannot capture localized geographic boundaries.
- **Severe Target Skewness:** Housing price distributions feature heavy right tails with high-value outliers and artificial census capping ($500,000+), causing standard regression loss functions to produce skewed residuals.
- **Feature Collinearity:** Total rooms, bedrooms, and population are heavily correlated with district density, requiring domain-informed ratio features rather than raw aggregate counts.
- **Deployment Gap:** Academic machine learning code frequently remains confined to static notebooks, lacking interactive interfaces suitable for end-user property assessment.

5. Objectives

- **Objective 1 (Data Ingestion):** Implement automated ingestion of the California Housing dataset (20,640 records) with reproducible disk caching.
- **Objective 2 (Exploratory Data Analysis):** Generate automated visual analytics including target distributions (raw vs. log-transformed) and correlation heatmaps.
- **Objective 3 (Feature Engineering):** Engineer domain-informed ratio features (`RoomsPerHousehold` and `PopPerHousehold`), apply log stabilization, and implement leak-free standard scaling.
- **Objective 4 (Cross-Validation Benchmarking):** Benchmark six distinct regression algorithms across 5-fold cross-validation under identical random seeds.
- **Objective 5 (Performance Evaluation):** Evaluate out-of-sample generalization using Test RMSE, MAE, and R^2, paired with residual analysis and feature importance extraction.
- **Objective 6 (Production Web Application):** Deliver a high-performance Streamlit dashboard featuring California city presets, model consensus visualization, interactive PyDeck mapping, and batch CSV processing.
- **Objective 7 (Unit Testing Suite):** Establish an automated test suite (`pytest`) verifying data integrity, feature transformations, split ratios, and prediction bounds.

6. Data set and Libraries required

6.1 Dataset: California Housing Dataset

The system is trained and evaluated on the benchmark California Housing dataset (Pace & Barry, 1997), encompassing 20,640 block-group districts derived from the 1990 U.S. Census:

Feature Name	Type	Range / Mean	Description & Content
`MedInc`	Float	0.50 – 15.00 (3.87)	Median income in block group (in tens of thousands of USD).
`HouseAge`	Float	1.0 – 52.0 (28.6)	Median age of residential buildings within the block (years).
`AveRooms`	Float	0.85 – 141.9 (5.43)	Average number of rooms per household dwelling.
`AveBedrms`	Float	0.33 – 34.07 (1.10)	Average number of bedrooms per household dwelling.
`Population`	Float	3.0 – 35,682 (1,425)	Total population residing within the census block group.
`AveOccup`	Float	0.69 – 1,243 (3.07)	Average number of household members residing together.
`Latitude`	Float	32.54° – 41.95° N	Block group geographic latitude coordinate.
`Longitude`	Float	-124.35° – -114.31° W	Block group geographic longitude coordinate.
`MedHouseVal`	Float	0.15 – 5.00 (2.07)	**Target:** Median house value (in $100,000 USD).

6.2 Engineered Features

To eliminate multicollinearity and capture demographic density:

- `RoomsPerHousehold`: AveRooms / max(HouseAge, 1.0) — normalized room availability per unit building age.
- `PopPerHousehold`: Population / max(AveOccup, 1.0) — residential occupancy saturation.
- `LogTarget`: ln(1 + MedHouseVal) — target variable variance stabilizer.

6.3 Libraries & Hardware Stack

- **Programming Language:** Python 3.14 / 3.10+ (64-bit)
- **Dashboard & Deployment:** Streamlit (≥ 1.35.0), PyDeck (≥ 0.9.0)
- **Scientific Computing:** NumPy (≥ 1.26.0), Pandas (≥ 2.0.0), SciPy (≥ 1.13.0)
- **Machine Learning & Boosting:** Scikit-Learn (≥ 1.4.0), XGBoost (≥ 1.7.0), Joblib (≥ 1.2.0)

- **Visualization:** Matplotlib (≥ 3.8.0), Seaborn (≥ 0.12.0)
- **Testing & Configuration:** Pytest (≥ 7.2.0), PyYAML (≥ 6.0), Tabulate (≥ 0.9.0)
- **Hardware Requirements:** Standard Intel Core i5/i7 or AMD Ryzen CPU, 8 GB RAM, 2 GB Disk Space.

7. Approach/ Methodology

7.1 End-to-End Pipeline (D.A.T.A. Protocol)

The project architecture guarantees complete separation between training and evaluation via the **D.A.T.A.** protocol:

1. **Discover (Data Ingestion):** Automated retrieval of raw dataset into immutable local storage at `data/raw/housing.csv`.
2. **Analyze (Feature Engineering & Scaling):** Automated generation of correlation heatmaps and target distributions; extraction of interaction ratios; natural log target transformation; fitting of `StandardScaler` strictly on training splits.
3. **Train (Model Training with 5-Fold CV):** Concurrent training of six regression models using 5-fold cross-validation; hyperparameters managed via centralized `config.yaml`. Serialization of fitted estimators to `models/*.joblib`.
4. **Assess (Model Assessment & Reporting):** Out-of-sample evaluation on the 20% holdout test set (4,128 block groups) computing RMSE, MAE, and R^2; generation of predicted-vs-actual scatter plots and automated reporting.

7.2 Mathematical Foundations

1. Multiple Linear Regression (Ordinary Least Squares):

$$\min_w ||y - Xw||_2^2 \Rightarrow \hat{w} = (X^T X)^{-1} X^T y$$

2. Ridge Regularization (L2 Penalty): Constrains coefficient norms to prevent collinear instability:

$$\min_w ||y - Xw||_2^2 + \alpha ||w||_2^2 \Rightarrow \hat{w} = (X^T X + \alpha I)^{-1} X^T y$$

3. Lasso Regularization (L1 Penalty): Drives uninformative feature coefficients strictly to zero:

$$\min_w \tfrac{1}{2} ||y - Xw||_2^2 + \alpha \sum_{j=1}^{p} |w_j|$$

4. Random Forest Regressor (Bagging): Aggregates B=100 independent decision trees:

$$\hat{y}(x) = (1 / B) \times \sum_{b=1}^{B} T_b(x)$$

5. XGBoost (Extreme Gradient Boosting): Optimizes regularized 2nd-order Taylor loss expansion:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} [g_i f_t(x_i) + \tfrac{1}{2} h_i f_t^2(x_i)] + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

6. Log-Target Inversion: Converts model log-predictions back into real USD price:

$$\text{Price}_{\text{USD}} = [\exp(\hat{y}_{\log}) - 1] \times 100{,}000$$

8. Implementation

The system is organized into a modular, production-ready Python package:

```
House-Price-Prediction/
├── config.yaml             # Centralized pipeline hyperparameters & paths
├── requirements.txt        # Pinned project dependencies
├── main.py                 # Single-command orchestration entrypoint
├── app.py                  # Production Streamlit analytical web dashboard
├── data/
│   ├── raw/housing.csv     # Immutable source CSV (20,640 records)
│   └── processed/          # Scaled and engineered feature matrices
├── models/
│   ├── scaler.joblib       # Serialized StandardScaler
│   ├── xgboost.joblib      # Champion model artifact (475 KB)
│   ├── random_forest.joblib# Serialized Random Forest artifact (10.7 MB)
│   ├── ridge.joblib        # Serialized Ridge artifact
│   ├── linear_regression.joblib
│   ├── lasso.joblib
│   ├── baseline.joblib
│   └── registry.md         # Markdown registry of cross-validation results
├── reports/
│   ├── final_report.md     # Auto-generated results summary
│   ├── Project_Report.pdf  # Formal compiled PDF submission
│   ├── Project_Report.html # Styled printable HTML report
│   ├── Project_Report.md   # Comprehensive SPPU mini project report
│   └── eda/                # 11 generated evaluation plots (PNG)
├── src/
│   ├── data/ingest.py      # Phase 1: Automated dataset discovery
│   ├── features/engineer.py# Phase 2: Ratios, log-transform, scaling, split
│   ├── models/train.py     # Phase 3: 5-fold CV & joblib serialization
│   └── evaluate/metrics.py # Phase 4: RMSE/MAE/R² metrics & plot generation
└── tests/
    ├── test_ingest.py      # Dataset schema and completeness tests
    ├── test_features.py    # Leakage, NaN, and split ratio unit tests
    └── test_models.py      # Prediction length, validity, and baseline benchmarks
```

Core Modular Components:

- `src/data/ingest.py`: Handles safe dataset acquisition and directory scaffolding.
- `src/features/engineer.py`: Implements ratio engineering, log-transformation, and StandardScaler fitting.
- `src/models/train.py`: Executes multi-model 5-fold cross-validation and records metrics into `models/registry.md`.
- `src/evaluate/metrics.py`: Computes test set RMSE, MAE, and R^2; generates residual and scatter plots.
- `app.py`: Delivers an interactive 4-tab Streamlit dashboard with real-time valuation, consensus comparison, interactive PyDeck GIS maps, and batch CSV valuation.

9. Results

9.1 Regression Benchmark Results

Evaluated on an identical 20% holdout test set (4,128 block groups) with 5-fold cross-validation on the training set:

Model / Algorithm	Test RMSE	Test MAE	Test R^2 Score	5-Fold CV RMSE	Train Time
XGBoost Regressor Best	**0.1424**	**0.0998**	**0.8391**	0.1425 ± 0.0013	4.1s
Random Forest Regressor	0.1611	0.1138	0.7941	0.1602 ± 0.0016	8.9s
Ridge Regression	0.2142	0.1603	0.6361	0.2086 ± 0.0043	3.4s
Linear Regression (OLS)	0.2142	0.1603	0.6361	0.2086 ± 0.0043	3.6s
Lasso Regression	0.2816	0.2254	0.3709	0.2802 ± 0.0031	0.3s
Median Baseline	0.3558	0.2879	-0.0042	0.3576 ± 0.0035	5.1s

9.2 Feature Importance & Driver Analysis

Analysis of tree-based split gains reveals the dominant real estate valuation drivers:

- **Median Income (`MedInc`, ~52.4% Importance):** Far and away the most dominant predictor. Higher neighborhood earning power directly correlates with elevated capital property values.
- **Geographic Coordinates (`Latitude` & `Longitude`, ~28.6% Combined):** Captures spatial proximity to the Pacific coast, Silicon Valley, and the Los Angeles basin.
- **Average Occupancy (`AveOccup`, ~9.2%):** High household density correlates with lower per-capita property valuations.
- **Building Age (`HouseAge`, ~5.8%):** Older historic neighborhoods in coastal metropolitan centers retain higher valuations.

9.3 Interactive Web Application Deployment

The trained models and preprocessing artifacts are deployed in a production-grade **Streamlit** application (`app.py`):

- **Tab 1 (Interactive Estimator):** California city presets (SF, LA, San Jose, San Diego, Sacramento, Fresno, Lake Tahoe) with instant multi-model consensus prediction bar charts and PyDeck maps.
- **Tab 2 (Model Leaderboard):** Live test set metrics, highlighted champion metrics, and feature importance bar plots.
- **Tab 3 (EDA Explorer):** Interactive geospatial scatter plot of California housing values and correlation heatmaps.
- **Tab 4 (Batch Inference):** Drag-and-drop CSV upload supporting multi-property valuation with one-click export.

10. Conclusion

The **California House Price Prediction & Real Estate Valuation System** successfully fulfills all curricular requirements of the SPPU Third Year Machine Learning laboratory:

1. **Demonstrated Superiority of Non-Linear Boosting:** Proved that **XGBoost** (R^2 = 0.8391, RMSE = 0.1424) vastly outperforms classical parametric regressions (R^2 = 0.6361) by modeling complex spatial autocorrelation.
2. **Mitigated Target Skewness:** Validated that natural log target transformation prevents high-value outliers from destabilizing squared-error gradients.
3. **Guaranteed Zero-Leakage Pipeline:** Enforced rigorous isolation between data discovery, feature scaling, 5-fold cross-validation, and final testing.
4. **Delivered Full-Stack Utility:** Transitioned serialized machine learning models into an interactive, high-performance web dashboard accessible to end-users.

References

1. Pace, R. K., & Barry, R. (1997). *Sparse Spatial Autoregressions*. Statistics & Probability Letters, 33(3), 291-297.
2. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
3. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
4. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer New York.

6. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer New York.
7. Savitribai Phule Pune University (SPPU). *Third Year Computer Engineering Syllabus & Machine Learning Laboratory Guidelines* (Course Code: 310246).